Universidade do Minho
Escola de Engenharia

Laboratórios de Informática III

-- Relatório Fase 1--

Grupo 32

Gabriel Matos Rodrigues a110987

Luís Pedro Matos Machado a110729

Luís Araújo Dias a111474

1. Introdução

No âmbito da unidade curricular de Laboratórios de Informática III, foi proposto a realização de um trabalho prático no intuito de desenvolver competências indispensáveis à implementação de programas de forma estruturada em linguagem C.

Este relatório diz respeito à primeira fase de desenvolvimento do trabalho prático, onde o principal objetivo foi adquirir conhecimentos ao nível de conceitos e técnicas de modularização e encapsulamento essenciais ao desenvolvimento do trabalho em C.

O tema do trabalho prático consiste num sistema baseado em aeroportos e voos, fundamentado na leitura e análise de ficheiros de dados (aeronaves, voos, reservas, passageiros, aeroportos).

2. Sistema

Após a análise do enunciado da fase 1 do trabalho prático proposto, verificamos que o desenvolvimento do nosso projeto teria de seguir uma determinada arquitetura. Deste modo, percebemos que a arquitetura base consistia numa divisão entre módulos de leitura de dados, interpretação de comandos, escrita de dados, execução das queries, gestores, entidades, estruturas de dados e módulos de utilidade.

Assim, no que diz respeito à arquitetura do nosso trabalho, o mesmo encontra-se dividido em duas diretorias principais, o src e o include. No include, temos os ficheiros de cabeçalho (.h), onde estão declaradas as funções de cada módulo e no src, encontram-se as implementações (.c) dos diferentes módulos presentes no include. Existem ainda diretorias auxiliares, como a dataset-fase1, formado pelo conjunto de datasets, a resultados e a resultados-esperados, e ainda a Makefile.

Quanto aos módulos da arquitetura do nosso projeto, optamos por dividi-lo nos seguintes módulos: main, gestor_entidades, entidades, parsers, queries e utils.

2.1. Main

O módulo Main é responsável por validar os diferentes argumentos, inicializar os gestores das entidades presentes no trabalho, carregar dos dados dos ficheiros csv, realizar a interpretação dos comandos do ficheiro input e mediante essa análise gerir qual a query que deve ser executada.

No que diz respeito à validação dos argumentos, o programa necessita de dois argumentos, o caminho para a pasta dos ficheiros csv e o caminho para o ficheiro input (inputs_fase1.txt), e a main valida a existência desses argumentos. Depois, para além da inicialização dos gestores e carregamento dos ficheiros csv, temos uma função importante, a leitura e execução das queries. É aqui que o ficheiro input é lido linha a linha, e para cada linha,

o comando é interpretado e a função executa a query correspondente, fornecendo –lhe os gestores que necessita e os diferentes dados csv necessários.

2.2. Entidades

O módulo entidades é responsável por criar as estruturas de cada uma das entidades, alocando memória para os diferentes campos de cada entidade, e por usa vez, gerir a memória usada na criação dessas mesmas estruturas, libertando-a de forma a não haver memory leaks. Estas entidades são depois geridas por cada pelo modulo gestor_entidades, tendo cada entidade o seu respetivo gestor e utilizadas posteriormente pelas diferentes queries, que necessitam de muitos dos seus dados.

2.3. Gestor_entidades

O módulo gestor_entidades é responsável por gerir as diferentes entidades do sistema, isto é, os aeroportos, aeronaves, voos, passageiros e reservas. As diversas entidades têm o seu próprio gestor, seguindo todos a mesma lógica de implementação.

Quanto à sua estrutura, cada um dos gestores é formado por uma GHashTable, e quanto às principais funções presentes no módulo, temos funções que criam e inicializam os gestores, libertam a memória associada a cada um deles e às próprias entidades, adicionam novas entidades aos gestores, verificam se entidades existem através de um identificador, removem entidades e ainda contam o número de elementos de uma determinada entidade existentes.

Existem ainda funções de validação, quer validações a nível sintático dos diferentes campos das diversas entidades, quer mesmo a nível lógico, ou seja, validar determinadas incongruências lógicas que devem ser tidas em conta ao utilizar o programa.

2.4. Parsers

Neste módulo Parsers cada uma das entidades tem uma função de parsing, em que, cada uma destas funções lê uma linha do ficheiro csv, divide-a nos diferentes campos associados a cada entidade, procede à validação desses diversos campos e cria a estrutura correspondente, por exemplo, voos, com os diversos campos validados e preenchidos.

Existe ainda funções de carregamento de dados, em que cada uma delas abre o ficheiro csv correspondente às diversas entidades, utilizam o parser necessário, registam as linhas inválidas num ficheiro de erros, permitindo identificar alguns erros de execução e posteriormente adicionam as entidades já validadas ao gestor que lhe corresponde.

2.5. Queries

No módulo queries, temos a implementação de cada uma das queries, 1, 2 e 3, em que cada uma delas interage diretamente com os gestores de entidades, acedendo aos diversos dados já validados. É depois no modulo main, que os comandos do input são lidos e é chamada a função correspondente do módulo das queries.

Este módulo é muito importante, já que são as queries que permitem através de outras funções que as auxiliam, responder a comandos do ficheiro input, ou seja, responder às solicitações que um determinado utilizador do programa faça.

2.6. Utils

O módulo utils é constituído por um conjunto de funções auxiliares gerais do projeto, ou seja, funções que não estão relacionadas especificamente com nenhum gestor, mas que são usadas em diferentes módulos da arquitetura do projeto.

Deste modo, neste módulo encontram-se funções de registo de erros, que permitem acompanhar as linhas consideradas inválidas durante o carregamento dos dados, e funções de validação, entre as quais, a validação dos códigos IATA, identificadores de voo, formato das listas csv, datas e horários.

Quanto à importância deste módulo, ele é muito utilizado pelos parsers e gestores, tendo um papel muito importante no asseguramento de que apenas dados válidos são integrados nas diversas estruturas. Sem este módulo, haveria a necessidade de repetir as mesmas funções em diversos módulos do projeto, para além de que a sua existência facilita a alteração, caso seja necessária, de alguma das funções de validação, já que pode ser feita apenas num único módulo.

Justificação da Arquitetura escolhida

A arquitetura do nosso projeto foi idealizada de forma a tornar o sistema em si organizado, fácil de manipular os dados e de simples percepção, tendo por base os conceitos de modularização e encapsulamento.

No que diz respeito à modularização esta consiste na divisão em diferentes módulos, cada um deles com funções específicas. Assim, justifica-se o facto de termos dividido o projeto em diferentes módulos, cada um deles com funções especificas, de forma a poder trabalhar em cada módulo individualmente, sem interferir nos restantes, facilitando a correção de eventuais erros que foram surgindo, bem como a adição ao longo do desenvolvimento, de novas funcionalidades.

Quanto ao encapsulamento, este está associado à proteção dos dados e das próprias estruturas e funções de cada um dos módulos. Cada um dos módulos, no nosso projeto, tem a sua própria implementação que é, de certa forma, escondida dos outros módulos, permitindo

apenas que os mesmos tenham acesso àquilo que realmente é necessário, o que torna o código do projeto mais seguro e acima de tudo, de mais fácil compreensão. Assim, com esta arquitetura, consideramos que o sistema em si se torna mais seguro e modular, e qualquer alteração que tenha de certa feita, bem como acrescentar novas funcionalidades é facilmente executada, não comprometendo as funcionalidades dos restantes módulos desenvolvidos no projeto.

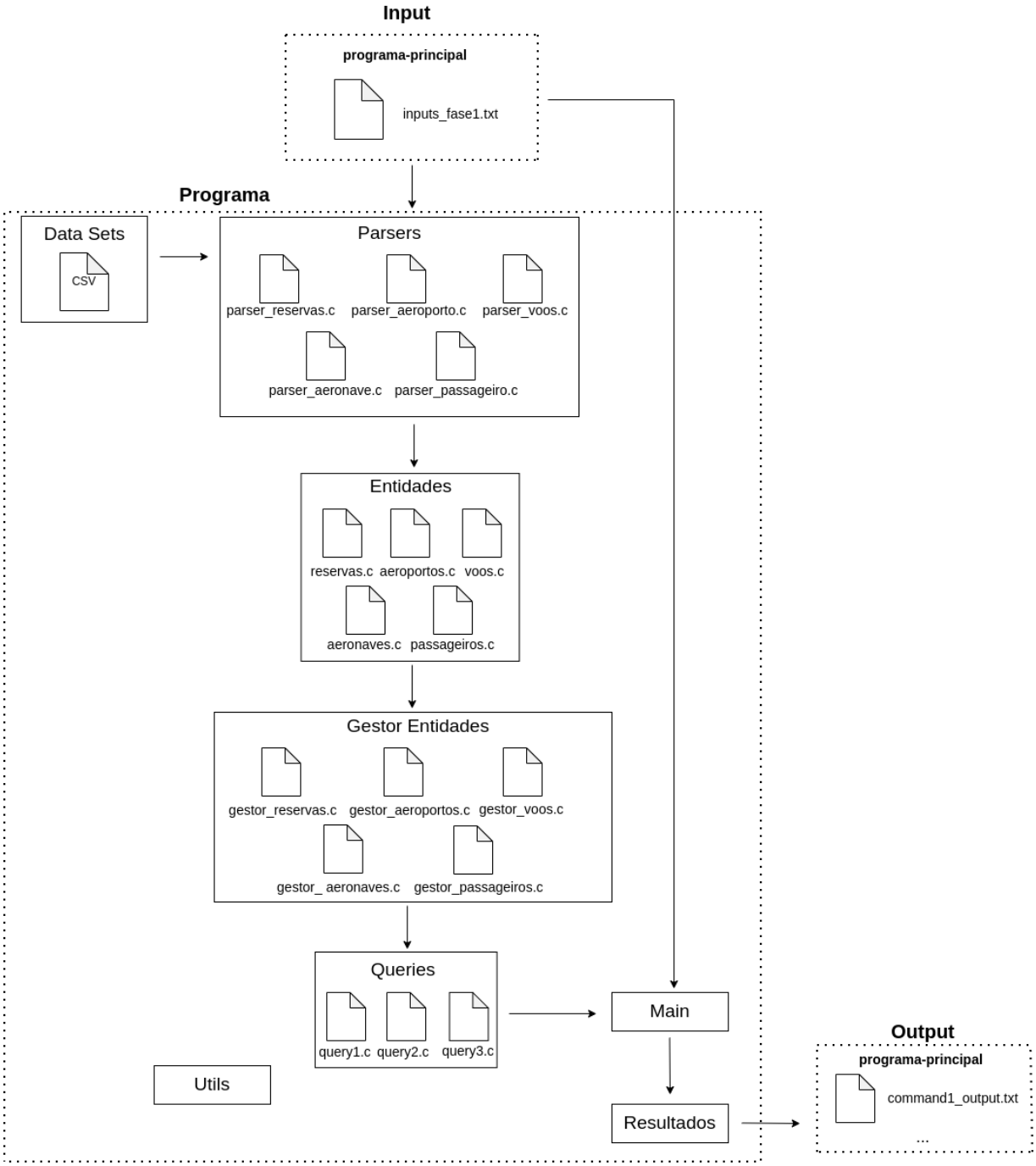

Figura1: Esquema representativo da Arquitetura do projeto

3. Estratégias usadas na implementação das Queries

Nesta primeira fase do trabalho prático, foi proposto um conjunto de queries, isto é, interrogações, as quais devem ser suportadas pela aplicação nesta primeira fase do trabalho.

Desta forma, para cada uma das queries (1, 2 e 3) foi adotada uma estratégia de implementação, de forma a que cada uma dessas mesmas queries executassem corretamente.

3.1. Query 1

A querie 1 tem como objetivo listar o resumo de um aeroporto, de acordo com o identificador recebido por argumento. Esta query recebe como argumento o identificador único do aeroporto e deve escrever no ficheiro output alguns campos desse aeroporto, tais como, o código, nome, cidade, país, e tipo do aeroporto.

Para implementar a query 1, usamos a tabela hash onde se encontram armazenados os aeroportos. A função percorre essa tabela, e usando a função g_hash_table_lookup, procura o aeroporto correspondente ao identificador único (código IATA) recebido inicialmente como argumento. Caso o aeroporto associado a esse código exista, escrevemos no ficheiro output, os diferentes campos da estrutura aeroporto pedidos, caso contrário, é escrita uma linha vazia no ficheiro output.

3.2. Query 2

A query 2 tem como objetivo escrever para o ficheiro output as Top N aeronaves com mais voos realizados. Esta querie recebe como argumento o número de aeronaves que devem aparecer no output e um filtro opcional, o filtro de fabricante, tendo em conta que quando existe, devem apenas ser consideradas aeronaves desse fabricante. No ficheiro output, deverão ser escritos alguns campos, tais como o aircraft, o manufacturer, o model e o flight count.

A nossa implementação da query 2, uma query 2 já mais complexa relativamente à query 1, divide –se em 4 fases. Inicialmente, começamos com a contagem de voos por aeronave. Para isso, criamos uma tabela hash auxiliar (contadoraVoos) que associa o identificador de uma aeronave ao número de voos realizados. Para isso ser possível, percorremos a tabela hash de voos, ignorando voos nulos e com estado “Cancelled”. No caso em que exista filtro de fabricante, apenas consideramos os voos de aeronaves cujo fabricante seja o do filtro, e posteriormente, para cada voo, incrementamos o contador da aeronave que lhe corresponde na tabela contadoraVoos. Nesta fase, o uso de uma tabela hash auxiliar foi importante, já que permitiu de forma eficiente contar o número de voos por aeronave perante um dataset bastante extenso.

Numa segunda fase, criamos uma estrutura ResultadosQ2 formada pelos campos aircraft_id, manufacturer, model e num_voos, com o objetivo de guardar de forma mais organizada os dados da contagem de voos feitos inicialmente. Os diferentes campos desta estrutura vão sendo preenchidos acedendo à tabela hash de aeronaves e ao número de voos disponível na tabela hash auxiliar.

De seguida, procedemos à fase de ordenar os resultados obtidos, usando para isso a função qsort. Esta função qsort usa uma outra função, a função comparar_q2, em que inicialmente a ordenação é feita por número de voos por ordem decrescente e, havendo empate, as aeronaves são ordenadas por ordem crescente do seu id.

Por último, é feita a escrita dos resultados obtidos no output, seguindo o formato anunciado inicialmente no enunciando da query 2. Consideramos esta implementação bastante eficiente, já que percorremos apenas uma vez a tabela hash de voos, referindo ainda o uso da função qsort que permitiu ordenar os resultados eficientemente.

3.3. Query 3

A query 3 tem como objetivo listar o aeroporto com mais partidas entre 2 datas. Esta query recebe como argumento as datas inicial e final e deve produzir no ficheiro output o aeroporto com mais partidas, incluindo o número total de partidas associado. Para a contabilização deste valor, serão apenas considerados voos cujo campo "origin" seja igual ao do aeroporto em questão.

Inicialmente, numa primeira implementação da querie 3, optamos por inicialmente percorrer a tabela hash de aeroportos, e depois, para cada aeroporto, percorrer a tabela hash de voos e verificar quantos deles tinham o campo "origin" igual ao do aeroporto em questão e se estavam dentro do intervalo das datas fornecidas como argumento. No entanto, após uma primeira execução verificamos que demorava imenso tempo a executar, devido a estas duas iterações seguidas sobre as tabelas hash, tendo em conta o tamanho dos ficheiros csv dos datasets usados.

Deste modo, alteramos a estrutura de implementação da querie 3, de forma a que apenas realizássemos uma única iteração sobre a tabela hash de voos. Inicialmente, criamos uma tabela hash auxiliar (numeroPartidas) que permitiu contar o número de partidas por cada aeroporto. A função percorre a tabela hash de voos, verifica se cada um dos voos em análise é válido (campos da estrutura voos em conformidade com aquilo que é imposto no enunciado da query 3) e se o voo for válido, o contador de partidas do aeroporto corresponde ao respetivo campo "origin" é incrementado. Esta tabela auxiliar tem como chave o código IATA do aeroporto e o valor é o número de partidas. É de salientar o uso de GPOINTER_TO_INT E GINT_TO_POINTER na fase de procura do aeroporto correspondente a um determinado campo "origin" que nos permitiu armazenar valores do tipo inteiro diretamente na tabela numeroPartidas, eliminado a necessidade de estar a fazer alocações adicionais.

Posteriormente, após a contagem terminar, percorremos novamente a tabela hash numeroPartidas, de forma a encontrar qual o aeroporto com maior número de partidas. Para isso usamos a variável numero_partidas_maximo, que guarda o número máximo de partidas e a variável melhor_aeroporto correspondente ao código. Caso haja um empate em que dois aeroportos têm o mesmo número de partidas, é escolhido o aeroporto cujo código seja menor em ordem lexicográfica. No final, com o código do aeroporto e já com o número de partidas final, percorremos a tabela hash de aeroportos e recolhemos os campos pedidos desse mesmo aeroporto, que depois são imprimidos no ficheiro output.

Com esta alteração, a execução da query 3 tornou-se muito mais eficiente, onde o tempo de execução diminuiu significativamente.

4. Discussão

De forma a avaliar o desempenho do nosso programa, efetuamos um conjunto de testes para cada query nas máquinas dos três elementos do grupo, tendo obtido os resultados mencionados na tabela abaixo. O tempo de cada uma das queries refere-se à média do tempo de 40 execuções, com os inputs do ficheiro inputs_fase1.txt e os datasets com erros. Para além dos tempos de execução das queries, em segundos, bem como o tempo total, encontram-se ainda mencionadas na tabela as características de cada uma das máquinas utilizadas, no que diz respeito ao processador e à memória RAM de cada uma delas.

Marca do Computador	Processador	Memória RAM	Tempo Query1	Tempo Query2	Tempo Query3	Tempo Total
HP Envy x360	13th Gen Intel® Core™ i7-1355U × 12	32 GB	0,000001s	0,324421s	0,318612s	25,7s
Asus Zenbook	Intel® Core™ i5-1035G1 × 8	8 GB	0,000002s	0,401688s	0,289856s	27,7s
Gigabyte G5 KF5	13th Gen Intel® Core™ i5-13500H × 16	16 GB	0,000001s	0,136015s	0,129280s	10,6s

De acordo com os resultados obtidos, descritos na tabela acima mencionada, verifica-se algumas diferenças nos tempos médios obtidos. O computador Gigabyte G5 KF5 obteve um melhor desempenho, quando comparado com os restantes, já que possui um processador mais rápido, mesmo tendo apenas 16 GB RAM, o que fez com que as queries fossem executadas num menor tempo. Quanto ao HP Envy x360, embora com 32 GB RAM, obteve um tempo de execução mais alto quando comparado com o Gigabyte, pois o seu processador é mais lento. O Asus Zenbook, foi o que obteve resultados mais altos (mais lento), já que possui um processador mais antigo, com menor desempenho, para além de ter apenas 8GB de RAM, o que faz com que a própria execução das queries, no que diz respeito ao acesso a estruturas de dados, seja mais lenta.

Em suma, verificamos que o fator mais relevante na existência de diferenças nos tempos, foi a velocidade do processador, e não a quantidade de memória RAM, já que mesmo o computador HP tendo mais memória RAM do que o Gigabyte, obteve um pior desempenho, devido ao seu processador mais lento.

5. Conclusão

O desenvolvimento da primeira fase do projeto inerente à unidade curricular Laboratórios de Informática III consistiu numa experiência enriquecedora a nível de conhecimentos de programação em C, destacando-se conceitos e técnicas de modularização e encapsulamento, que se evidenciaram muito importantes em toda a organização do código.

Além disso, é ainda de referir que enfrentamos algumas dificuldades ao longo do desenvolvimento do código, tais como a implementação das queries 2 e 3, onde houve necessidade de perceber quais soluções poderíamos adotar para ter implementações eficientes a nível de execução, já que os datasets usados eram bastante extensos.

É de mencionar ainda a existência de algumas dificuldades na gestão de memória, onde, por vezes, era alocada memória para determinadas estruturas que depois não era libertada, tendo a ferramenta valgrind constituído um mecanismo essencial na resolução destes problemas.

Por fim, consideramos ter tido um bom desempenho nesta primeira fase do projeto, tendo implementado as diversas funcionalidades que deveriam estar presentes nesta primeira parte, não deixando de ressaltar a consciência de que poderão existir melhorias na próxima fase do projeto